MASTER OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE SYSTEMS

Pattern Recognition Systems

Project Report

Multimodal Ceramic Glaze Retrieval Engine

Group 45

Team members

No.	Full Name	Matriculation ID
1	Lim Hao	A0354214M
2	Pang Claire Fuiyi	A0357738L
3	Polthai Nakhin	A0355086X
4	Le Duc Nghia	A0356261E

TABLE OF CONTENTS

MASTER OF TECHNOLOGY IN ARTIFICIAL INTELLIGENCE SYSTEMS **0**

Multimodal Ceramic Glaze Retrieval Engine **1**

TABLE OF CONTENTS **2**

LIST OF FIGURES **4**

LIST OF TABLES **6**

1. Project Background **7**

2. Market Research **9**

2.1 Digital Repositories 9

2.2 Market Drivers 9

3. Project Overview/ Executive Summary **11**

3.1 Project Introduction 11

3.2 Project scope 11

3.3 Project limitations 11

4. System Design and Architecture **13**

4.1 Data Sources 13

4.2 Data Processing 15

4.2.1 Data Dictionary 17

4.3 Image Selection & Segmentation 20

4.3.1 Grounding DINO: text-prompted detection 23

4.3.2 OpenCLIP: verification and rejection 23

4.3.3 SAM 2.1: pixel-level segmentation 24

4.3.4 Object extraction and saving 25

4.3.5 Colour Extraction and Region Analysis 25

4.4 Training Process 28

4.4.1 Shared Latent Space 29

4.4.2 k-means Clustering 30

4.4.3 Experimentation 30

4.4.4 K-means Result Diversification 31

4.5 Recipe Retrieval..... 31
4.5.1 Retrieval Weight Experimentation and Results..... 32
4.6 User facing app..... 36
4.6.1 Image upload and color extraction pipeline..... 38
5. Evaluation Metrics..... 44
5.1 Real-Photo Retrieval Evaluation..... 44
5.1.1 Hit@5..... 44
5.1.2 Precision@5..... 44
5.1.3 Recall@5..... 45
5.1.4 Results and Discussion..... 45
5.2 Colour-Matching Evaluation..... 46
5.3 Limitations and Future Evaluation..... 47
6. Mapping of System Functionalities against knowledge, techniques and skills of modular courses.... 49
Appendix A. Segmentation Steps and Thresholds..... 50
Appendix B. Model Versions and Processing Parameters..... 51
Appendix C: User Installation Guide..... 52
References..... 54
AI Tool Declaration..... 56

LIST OF FIGURES

Figure 1. Examples of ceramic glaze appearances: (a) pink-purple crystalline glaze on a ceramic bowl; (b) yellow glossy glaze on a ceramic test tile..... 7

Figure 2. Logos of two online ceramic glaze resources: (a) Glazy, an open-source ceramic glaze recipe database; (b) Digitalfire, a platform for ceramic glaze chemistry and technical information..... 8

Figure 3. System architecture diagram.....13

Figure 4. The thumbnail photo for recipe 16050..... 15

Figure 5. Ceramic glaze image processing and dominant colour extraction pipeline: (a) sky-blue ceramic test tile (#84B9C5); (b) blue-grey ceramic mug (#698F90). Each example illustrates the original photograph, background removal through image segmentation, and extraction of the dominant glaze colour.....16

Figure 6. Image processing pipeline from photograph to object library.....21

Band ① crops the object automatically, band ② describes it with four feature vectors and band ③ is the human review before indexing. All thumbnails are real outputs for 18.jpg, except the correction example, which is photograph 6051.....21

Figure 7. Segmentation steps of CeramicSegmenterV3 for 18.jpg.....22

Figure 8. Detection on photograph 18.jpg..... 24

Figure 9. SAM 2.1 candidate masks and the saved cut-out..... 25

Figure 10. Colour regions of object 18_1..... 27

Figure 11. Stored ColorThief palette of 18_1..... 28

Figure 12. Sample glaze colour queries used for retrieval weight experiments: (a) midnight blue (#0931AA), glossy, Cone 6; (b) dusty pink (#C68896), glossy, Cone 6.....33

Figure 13. Results when only the latent space cosine similarity is used in retrieval for a midnight blue glossy cone 6..... 34

Figure 14. Results when only the latent space cosine similarity is used in retrieval for a dusty pink glossy cone 6..... 34

Figure 15. Results when the retrieval weights for midnight blue glossy cone 6 is 70/10/10/10..... 34

Figure 16. Results when only the retrieval weights for dusty pink glossy cone 6 is 70/10/10/10.....35

Figure 17. Results when the retrieval weights for midnight blue glossy cone 6 is 40/40/10/10..... 35

Figure 18. Results when the retrieval weights for pale pink glossy cone 6 is 40/40/10/10.....36

Figure 19. Streamlit application model availability diagnostics, showing the detection of cached pretrained

model weights for Grounding DINO, SAM 2.1, and MiniLM, alongside OpenCLIP model verification..... 36

Figure 20. Streamlit user interface of the Multimodal Ceramic Glaze Retrieval Engine, showing the image upload, segmented object selection, and glaze recipe search components..... 37

Figure 21. Jupyter Notebook implementation of the image input process, demonstrating image path configuration, file loading, and visualization of a sample ceramic glaze photograph..... 38

Figure 22. Streamlit app: Segmentation and color extraction results when only a single object is detected... 39

Figure 23. Streamlit app: Segmentation and color extraction results when more than 1 object is detected in the input photo. Object 1 is selected, and the color results from Object 1 are shown..... 39

Figure 24. Streamlit app: Segmentation and color extraction results when more than 1 object is detected in the input photo. Object 2 is selected, and the color results from Object 2 are shown..... 39

Figure 25. Notebook app: code snippet for color extraction.....40

Figure 26. Notebook app: code snippet for color retrieving..... 40

Figure 27. Streamlit app: Retrieved results for the sample input image.....41

Figure 28. Streamlit app: Retrieved results with diversification for the sample input image. The highest ranking recipes with up to 2 results from each cluster are retrieved.....42

Figure 29. Notebook: Retrieved results for the sample input image (regular, non-diversified)..... 43

Figure 30. Notebook: Retrieved results for the sample input image (diversified using k-means cluster labels).. 43

Figure 31. Colour-matching evaluation of ceramic glaze recipe 794807 against the requested colour (#6B7456), showing (a) the original photograph with the detected ceramic object, (b) the segmented glaze object, and (c) the pixel-level colour similarity visualization using the CIEDE2000 (ΔE00) metric.....47

.....

LIST OF TABLES

Table 1: Training Dataset Data Dictionary..... 17

Table 2. The five most relevant colour regions of 18_1..... 27

Table 3. Execution order and settings of CeramicSegmenterV3..... 51

Table 4. Model versions and processing parameters of the image pathway..... 52

1. Project Background

A ceramic glaze is a glass formula that melts on and fuses to a ceramic clay body, sealing and waterproofing the porous clay body, increases its durability and provides an aesthetical dimension to the overall object. Glazes play an important role in modern ceramics, allowing ceramicists an additional avenue to express their creativity and bringing about an exponential number of possible designs.

Figure 1. Examples of ceramic glaze appearances: (a) pink-purple crystalline glaze on a ceramic bowl; (b) yellow glossy glaze on a ceramic test tile.

Most ceramic glazes consist of oxides that fulfill three main functions: a glass former, a melting agent, and a stabilizer. On top of these, a variety of materials, including colorants and crystallisers, are also commonly found in glazes. With innumerable possible combinations of materials leading to an equally dizzying number of possible results, glaze formulation testing presents a pluralistic problem requiring exhaustive empirical testing or extensive domain expertise. Glaze formulation has become a part of the craft that many ceramicists avoid diving too deep into.

One way for ceramicists to learn about new glaze recipes is through ceramics books, and another channel is online databases like Glazy and Digitalfire Insight, which feature recipes posted by individual ceramicists as well as the collection of established and stable ceramicists. These open-source repositories have widely increased the access to glaze recipes and materials

information, and allow their users to search through tens of thousands of individual recipes uploaded by fellow users around the world. The high level of availability encourages collaborative glaze research and enriches various stratum of practitioners.

Figure 2. Logos of two online ceramic glaze resources: (a) Glazy, an open-source ceramic glaze recipe database; (b) Digitalfire, a platform for ceramic glaze chemistry and technical information.

However, prevailing methods of retrieving glaze recipes from digital repositories are restricted to querying mechanisms with multiple optional, Boolean keyword matching, or exact stoichiometric inputs that present convoluted options to users. At the same time, this complex process fails to capture the nuanced multi-dimensionality nature of glaze behaviours that users may have in mind.

2. Market Research

The global glaze market, encompassing the crafts industry as well as industrial and construction-adjacent sectors, is forecasted to expand to up to USD 9.1 billion by 2035, driven by demand across the entire market. This highlights the potential magnitude of the market and that any new technologies could see significant returns.

2.1 Digital Repositories

The management of and access to digital repositories are represented by a small number of hubs. Glazy.org is the leading database with the most significant number of active users and contributors. It hosts tens of thousands of recipes for not just glazes but clay bodies and underglazes as well. The raw dataset is hosted on GitHub by its creator Derek Au, and is seeded from legacy datasets from established ceramicists' long-standing databases.

OpenGlaze is an open-source glaze management software, allowing ceramicists to calculate raw materials, analyse Unity Molecular Formula, estimate Coefficient of Thermal Expansion, manage their own database of recipes, and access an AI-powered glaze consultation engine. The calculator automates many experiments that would have been done manually, reducing the amount of raw materials needed to achieve desired results.

DigitalFire is the longest-running platform, created by Tony Hansen in the late 1970s first as a desktop glaze chemistry calculator. It currently functions as an cross-platform environment to cache glaze recipes, record test results and firing schedules, and various supplementary data. It also hosts a limited range of glaze recipes for demonstrative and educational purposes.

2.2 Market Drivers

All glaze recipes rely on raw materials acquired from mines or other natural environments, which inevitably exposes glaze producers and ceramicists to supply chain disruptions when these natural materials are depleted. For example, a major and in-demand line of Shino glazes by AMACO had to be discontinued in November 2026 due disruptions to their supply of an undisclosed proprietary material. Being able to search for suitable and accessible alternative glaze recipes or raw materials when such events occur is critical, especially for manufacturers with large volumes of orders.

Various players in the glaze industry are also subjected to changes in demand trends, like an increase in environmental awareness would lead to a higher demand for glaze recipes with less-destructive raw materials and more sustainable production processes.

3. Project Overview/ Executive Summary

3.1 Project Introduction

To address the issue of plurality in glaze recipes factors and bridge the gap between users' intended glaze appearance and available glaze recipes on digital repositories, our team developed a Multi-Modal Data Fusion Retrieval Engine. This system maps low-level inputs to quantitative glaze appearance and behaviour in its end-to-end retrieval architecture, enabling users to efficiently and effortlessly query optimal glaze formulations that satisfy both aesthetic criteria and structural stability.

3.2 Project scope

This solution encompasses three main functionalities: glaze analysis and classification, data standardisation, and recipe retrieval. It also includes an interactive user interface for query inputs and image capturing. The system pipeline has both Computer Vision and Machine Learning functionalities: utilizing Grounded-SAM and CLIP models for segmentation, classification, and semantic tagging, with specialized modules like Retinex algorithms and un-reflection utilities for image enhancement. These modules work together with a retrieval engine, relying on vector embeddings, structured metadata, and image evaluation metrics for high-speed similarity searches, supported by cross-platform environment configurations and interactive application scripts.

3.3 Project limitations

The limitations of this system include both practical and technical constraints. From a high-level perspective, the tool struggles with wide variations in pottery photography - such as tight camera angles, unique surface textures, and low image quality - that could lead to inaccurate classifications or mismatched retrieval results. This could be addressed by adding a detailed instructional demonstration on a more polished version of the user interface. The system requires a manual update to its data pipeline to adapt to novel glaze formulas due to its reliance on pre-computed vector databases and static catalog metadata. Potential future improvements to address this would be migrating to a light-weight vector database and pairing it with an automated ingestion service or

API endpoint. Another would be that the currently applied machine learning models are computationally heavy and require significant GPU resources to run. Future potential implementation, depending on use case, could include half-precision models or lighter architectures, and routing image tasks through a tasks queue.

4. System Design and Architecture

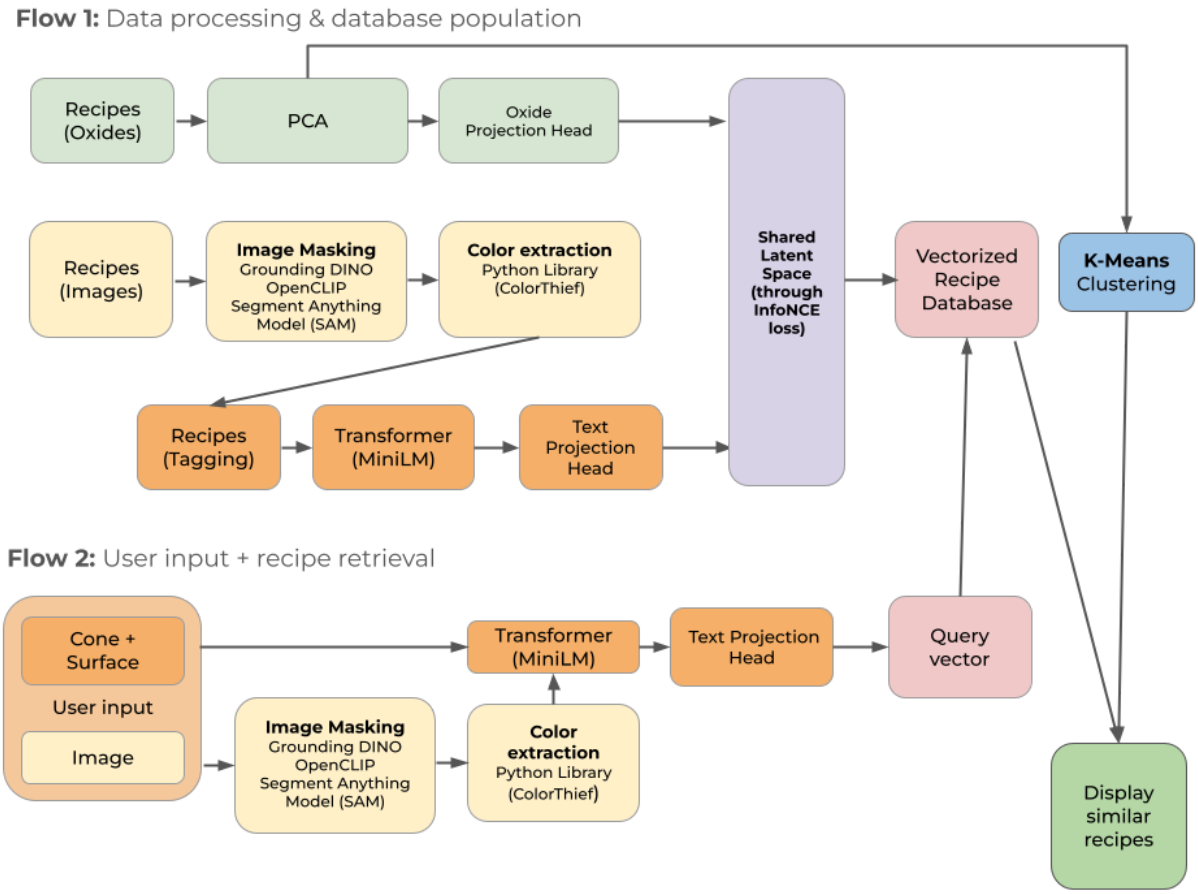


Figure 3. System architecture diagram.

4.1 Data Sources

We are using the open source dataset from glazy.org. Periodically, the site owner uploads a data export to Github. The export we have is from 31 May 2026.

Our dataset consists of images, ingredient formulation, oxide composition, and tagging.

The recipe data format is in a YAML file. A sample recipe looks as below:

```
ID: 19438
 Name: 'Tsaber''s Crackle Celadon'
 Other Names: 'Guan Crackle Celadon'
 Created By:
   Name: 'Alexander Kuttel'
   ID: 968
 Date Created: '2018-05-22'
 Date Modified: '2025-07-10'
 Type: 'Recipe'
 State: 'Production'
 Subtype: 'Glaze - Iron - Celadon - Blue'
 Cone: '10 - 12'
 Surface: 'Glossy'
 Transparency: 'Transparent'
 Atmospheres: ['Reduction', 'Salt & Soda']
 Description: 'This glaze needs heat. It forms a beautiful glossy ice blue when fired
nice and hot (or fired twice), otherwise it won''t fully form a glass and will come out a
semi-matte if you only hit cone 9. If you''re having trouble getting it to fuse even with
higher temps, add more Whiting or Barium Carbonate to bring the glaze into a better flux
ratio.'
 Ingredients:
   -
     ID: 16050
     Name: 'Nepheline Syenite A270'
     Percentage: 53
   -
     ID: 15400
     Name: 'Silica'
     Percentage: 29.5
   -
     ID: 15457
     Name: 'Whiting'
     Percentage: 15.5
   -
     ID: 15072
     Name: 'Bentonite'
     Percentage: 5
   -
     ID: 15062
     Name: 'Barium Carbonate'
     Percentage: 5
     Additional: true
```

```
  -
    ID: 15387
    Name: 'Red Iron Oxide'
    Percentage: 0.6
    Additional: true
Percent Analysis:
  SiO2: 59.2748
  Al2O3: 12.4436
  Na2O: 5.2145
  K2O: 2.4133
  MgO: 0.1023
  CaO: 8.2435
  BaO: 3.5755
  Fe2O3: 0.7257
  LOI: 8.0262
```

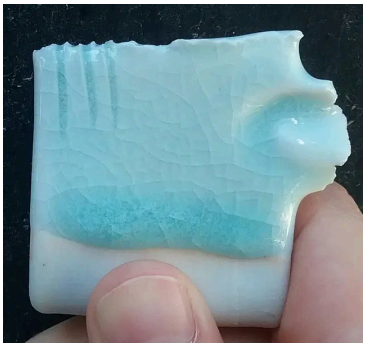



Figure 4. The thumbnail photo for recipe 16050.

The image consists of the main thumbnail photo, available only if the user who uploaded the recipe also uploaded a photo.

Example link to above recipe on Glazy

4.2 Data Processing

For the row-level data, we first converted the data to CSV.

The raw dataset from the Glazy export contains 43,084 recipes.

We filtered the dataset to only include recipes that:

1. Have a photo (7,189 recipes)
2. Correspond to a glaze recipe (not a clay body)

Our final dataset consists of **6,723 recipes**.

Because the recipe tags are dependent upon the user selecting those fields when they upload the recipe, the color value is incomplete. The user uploaded image typically consists of a sample glaze tile or a full pottery object using the glaze. The thumbnail images were run through our image segmentation pipeline to retrieve cropped versions with only the pottery piece or glaze sample tile. We then ran it through our color extraction script to extract the dominant hex code, with a corresponding color name. A one-time pass of all the training images for image segmentation and color extraction on a personal computer took around 12 hours to run for all recipes.

More details on the image segmentation pipeline and color extraction can be found in the following sections.

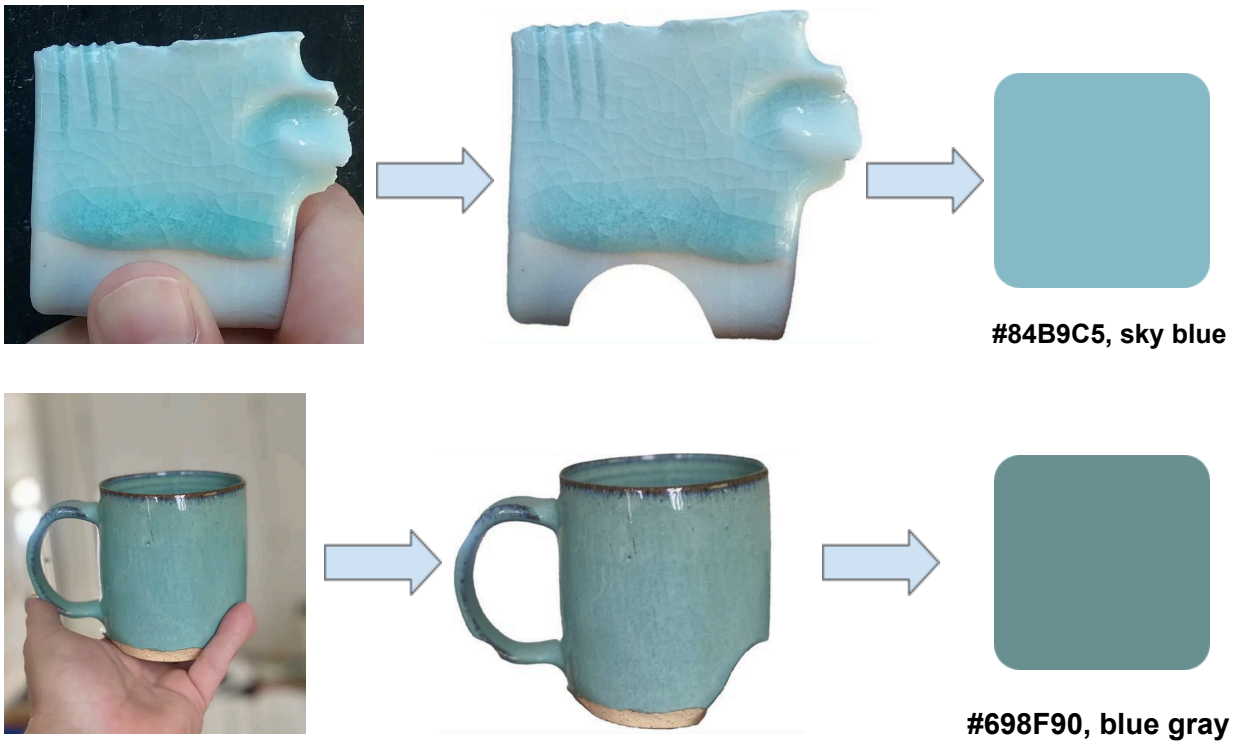


Figure 5. Ceramic glaze image processing and dominant colour extraction pipeline: (a) sky-blue ceramic test tile (#84B9C5); (b) blue-grey ceramic mug (#698F90). Each example illustrates the original photograph, background removal through image segmentation, and extraction of the dominant glaze colour.

Various other data cleaning and processing checks were completed, such as:

- Standardising firing cones with decimals, such as 5 ½ - 7 → 5.5 - 7
- Glaze subtypes with a label as “clear” by the user has NULL hex code and “clear” for color name (results from color extraction pipeline overridden)
- Merged hex code and color name from image segmentation and color extraction pipeline

The complete script and intermediate CSVs used to create the final training dataset can be found in training/dataset_creation.

4.2.1 Data Dictionary

Below are the columns in our training dataset:

Table 1: Training Dataset Data Dictionary

Column Name	Sample value	Column Definition
recipe_id	19438	The unique ID for a recipe
image_path	https://cdn.glazy.org/public/uploads/recipes/38/l_19438.5b3ace8172c28.jpg	The URL path for the thumbnail image. Not used for training.
color_name	sky blue	The color name based on the dominant hex color that was extracted from the thumbnail image
hex_color	#84B9C5	The most dominant hex color extracted from the segmented pottery thumbnail image
Cone	10 - 12	Firing temperature
Surface	Glossy	type of surface of glaze Possible values: Glossy Matte Satin Satin-matte Semi-glossy Semi-matte

		Smooth mate Stony matte Dry matte
Transparency	Transparent	How much of underlying clay body is visible through the glaze Possible values: Opaque Semi-opaque Translucent Transparent
Atmospheres	Reduction; Salt & Soda	The type of environment during a ceramic firing, affects how glazes and clay bodies react, and determines glaze formation e.g. oxidation, reduction, neutral, atmospheric
SiO2	59.2748	These columns correspond to the oxide makeup of the recipe. Generally, the oxides determine glaze appearance by affecting the colour, structure, melting and surface of glazes
Al2O3	12.4436	
CaO	8.2435	
K2O	2.4133	
Na2O	5.2145	
Fe2O3	0.7257	
MgO	0.1023	
TiO2		
B2O3		
P2O5		
ZnO		
CuO		
CoO		
SrO		

Li2O		
ZrO2		
SnO2		
BaO	3.5755	
Cr2O3		
MnO		
MnO2		
FeO		
NiO		
PbO		
F		
V2O5		

4.3 Image Selection & Segmentation

The image pathway turns each Glazy recipe photograph [1] into one or more segmented ceramic objects and then describes every object in a form that a search can compare: a set of colour regions, a three-colour palette, a glaze-appearance vector, a shape descriptor and probabilities for six visual attributes. The same code runs in two places. Offline, color name and dominant hex code is used as input for the text stream. Online, it processes whatever photograph a user drops into Glaze Search.

A single photograph is followed through this section: 18.jpg, the photograph of Glazy recipe 18, *Amber Celadon McKenzie Smith*, a glossy amber iron glaze on a test tile held between two fingers. The intermediate results are not illustrations. They were produced by running the project's own segmenter on this photograph with the pinned model weights, and the resulting mask was identical to the mask stored in the dataset (intersection over union, IoU = 1.000).

Figure 1 gives the whole route on one page. Band ① crops the ceramic object out of the photograph (Section 4.3.1). Band ② turns the cut-out into four feature vectors, for glaze appearance, shape, surface attributes and colour (Sections 4.3.2 and 4.3.3). Band ③ is the human check that corrected masks and labels before the objects were indexed (Section 4.3.4). The numbered badges follow the order in which the steps run.

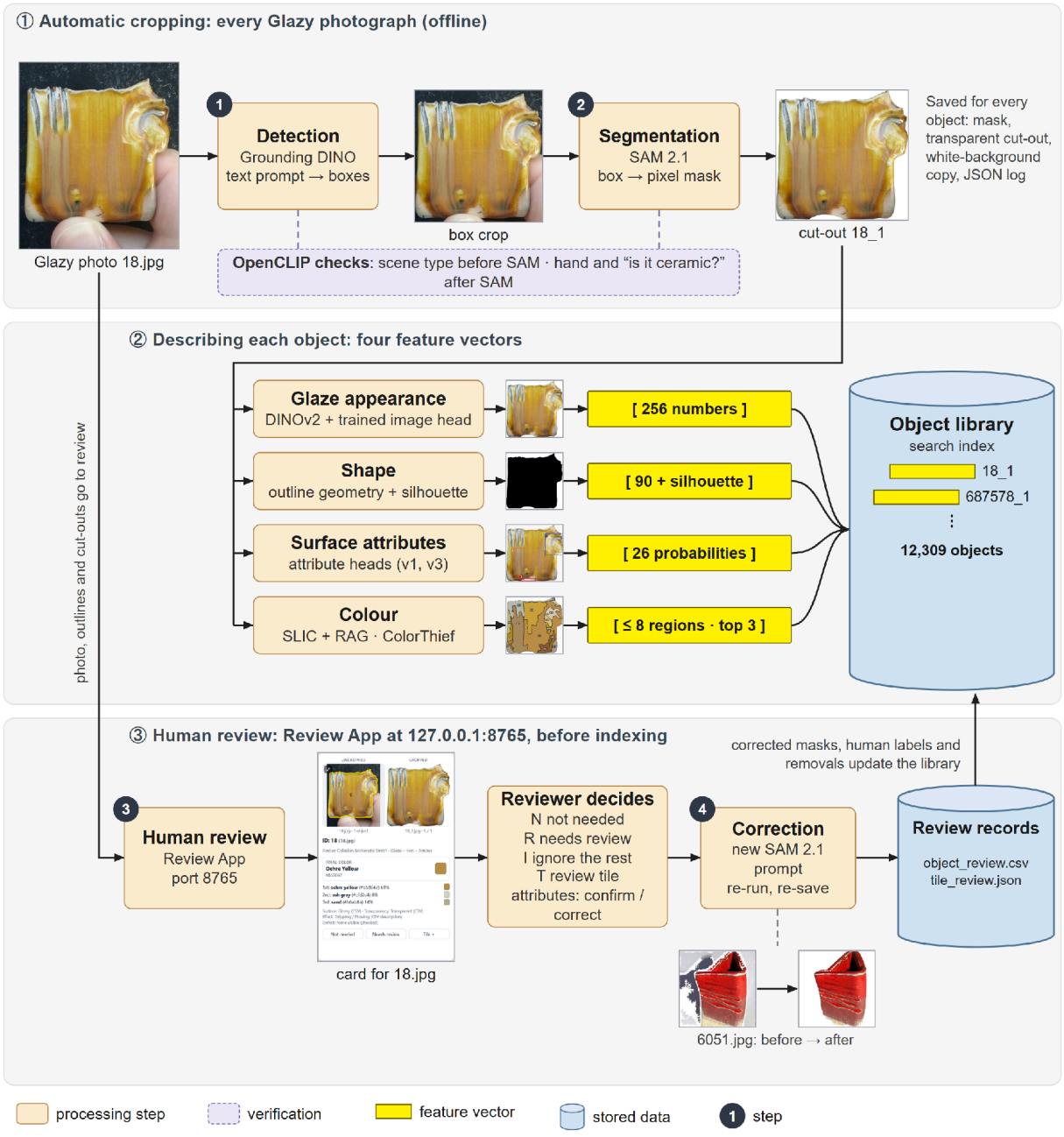


Figure 6. Image processing pipeline from photograph to object library.

Band ① crops the object automatically, band ② describes it with four feature vectors and band ③ is the human review before indexing. All thumbnails are real outputs for 18.jpg, except the correction example, which is photograph 6051.

Glazy photographs are rarely clean product shots. Test tiles are often held in a hand, several tiles may lie on one board, and recipe labels, tape or a kiln shelf often share the frame. Segmentation therefore has to do two things at once: find every ceramic piece and leave out everything else. It is implemented in the class `CeramicSegmenterV3` (`glazeprep/segmentation/ceramic_v3.py`), with all thresholds collected in the `V3Params` data class. Three pretrained models do the work, all used frozen with their downloaded weights: Grounding DINO [3] for text-prompted detection, SAM 2.1 [4] for pixel-accurate masks and OpenCLIP [5] for checking what a region actually shows. The model versions are pinned in `scripts/setup_models.py` and loaded offline from the local model cache (Appendix B).

The order in which these models run was read from the `__call__` method rather than assumed. It is not a simple detect, verify, segment chain: CLIP is consulted on the whole photograph before SAM runs, and twice more on individual regions afterwards. Figure 2 follows 18.jpg through the nine steps in the order in which they execute. The thresholds of every step are listed in Table 3 in Appendix A.

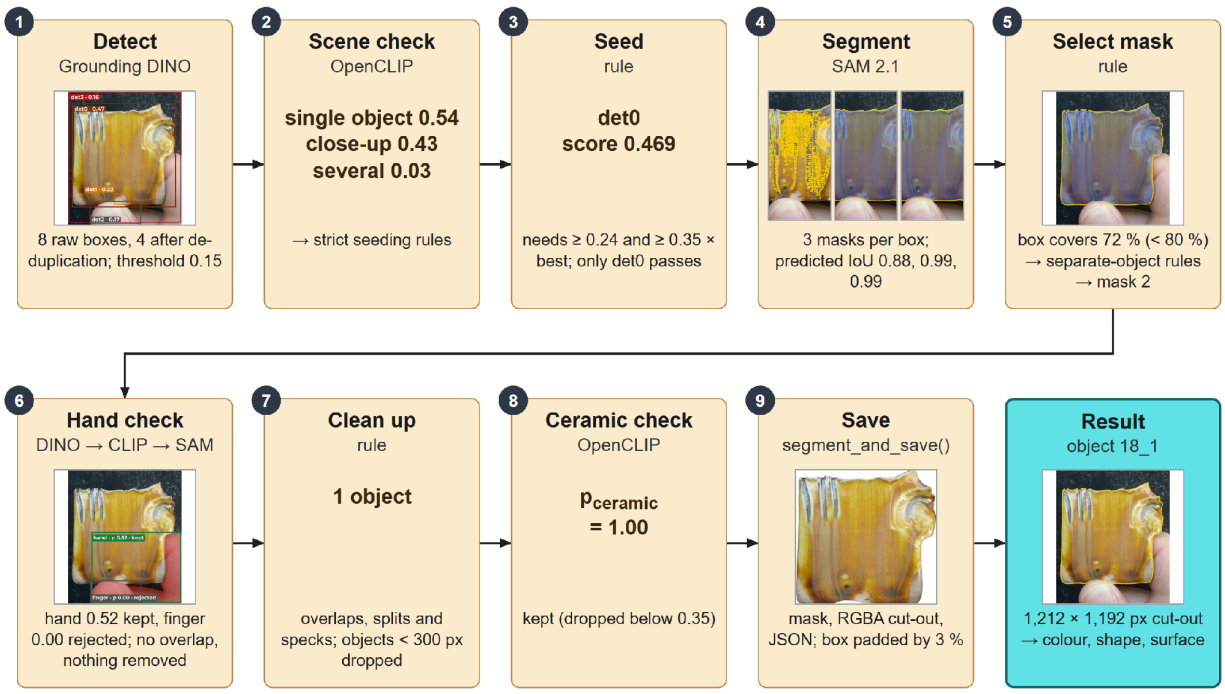


Figure 7. Segmentation steps of CeramicSegmenterV3 for 18.jpg.

Each card gives the model or rule, the result for this photograph and the main setting, in the order of `__call__`.

4.3.1 Grounding DINO: text-prompted detection

Input. The RGB photograph and a prompt of ten phrases: "ceramic object. pottery piece. ceramic test tile. ceramic test sample. ceramic fragment. pottery shard. broken ceramic piece. ceramic plate. ceramic bowl. glazed pottery." (`OBJECT_PROMPT` in `grounded_sam.py`, the default prompt of `V3Params`). *Processing.* Grounding DINO (IDEA-Research/grounding-dino-base) encodes the image with a Swin Transformer and the prompt with BERT, fuses the two streams and decodes a fixed set of candidate boxes [3]. Every box is scored against every prompt token and returned when its best phrase passes the threshold. *Output.* Boxes, scores and the matching phrase. *Next stage.* The surviving boxes become the prompts for SAM 2.1.

The threshold is deliberately low (0.15). A faint tile that is missed here cannot be recovered later, whereas an extra box can still be rejected by the rules and by CLIP. On 18.jpg the detector returned eight raw boxes, and de-duplication left four (Figure 3b). Only det0 (score 0.469, "ceramic test sample / glazed pottery") was strong enough to seed an object. det1 (0.219) lay inside det0 and was merged into it, det2 (0.172) on the thumb was too weak, and det3 (0.164, the whole frame read as "plate") was rejected as background because 59 % of its mask touched the image border.

4.3.2 OpenCLIP: verification and rejection

Input. Either the whole photograph or a masked crop of one region on a neutral grey background, together with fixed lists of sentences. *Processing.* OpenCLIP ViT-B/32 [5], a CLIP model [6] trained on LAION-2B (`laion2b_s34b_b79k`), embeds the image and every sentence; a softmax over the cosine similarities turns them into probabilities, which are summed per group. Three sentence sets are used: scene sentences (close-up, single object, several objects, non-ceramic), hand sentences (three positive, seven negative) and the ceramic check (12 ceramic against 18 non-ceramic sentences such as "a wooden table", "a strip of masking tape" or "a printed label"). *Output.* Scene probabilities, p_{hand} for each hand box and $p_{ceramic}$ for each object. *Next stage.* The scene decides how strict seeding is, p_{hand} decides whether a hand is cut away, and $p_{ceramic}$ removes tables, paper and boards before the objects are saved.

For 18.jpg the scene probabilities were 0.54 single object, 0.43 close-up, 0.03 several objects and 0.00 non-ceramic, so the stricter single-object seeding applied. Grounding DINO also found a

"hand" box and a "finger" box. CLIP confirmed the hand ($p_{hand} = 0.52$) and rejected the finger (0.00), as shown in Figure 3c. The hand mask that SAM drew for the confirmed box did not overlap the tile, so nothing was removed, and the tile itself scored $p_{ceramic} = 1.00$.

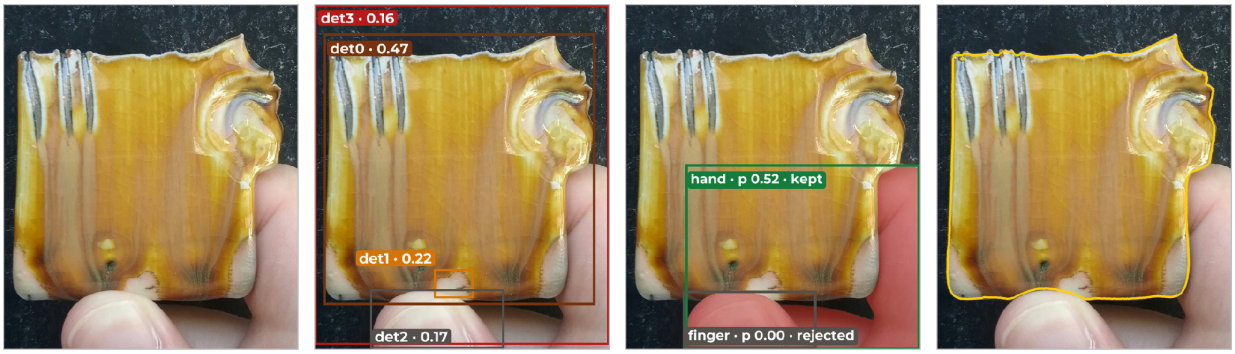


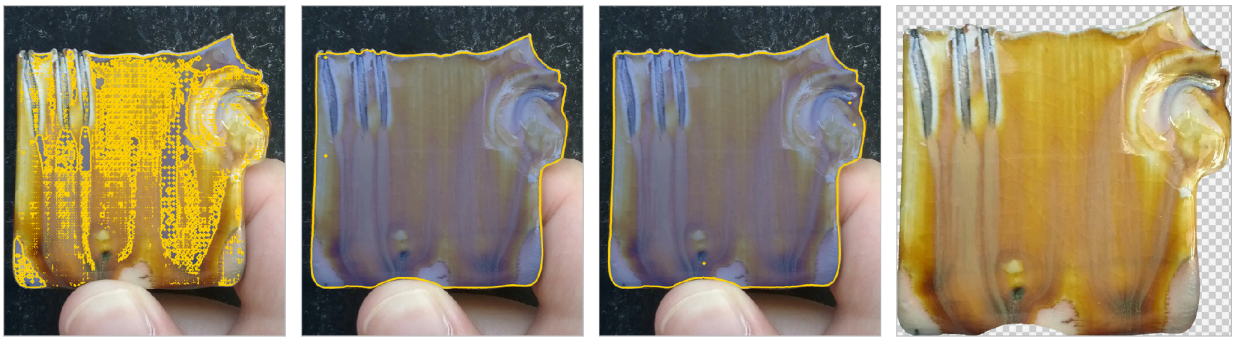
Figure 8. Detection on photograph **18.jpg.**

(a) Original photograph; (b) Grounding DINO boxes after de-duplication, with scores; (c) hand check, where CLIP confirms the hand box ($p_{hand} = 0.52$) and rejects the finger box (0.00); (d) outline of the final object. Output of `CeramicSegmenterV3` runs on the photograph.

4.3.3 SAM 2.1: pixel-level segmentation

Input. The photograph and one box prompt. *Processing.* SAM 2.1 [4], the successor of the Segment Anything Model [7], embeds the photograph once at 1,024 × 1,024 pixels with its Hiera-L image encoder (facebook/sam2.1-hiera-large). The prompt encoder turns each box into tokens and the mask decoder returns three alternative masks per box, each with a predicted IoU (the model's own quality estimate) and an object score. *Output.* Three candidate masks per prompt. *Next stage.* The selection rules of step 5 keep one candidate, or combine several, as the object instances.

Figure 4 shows the three candidates for det0. Mask 1 (predicted IoU 0.883, 24 % of the image) covers only the shiny streaks. Masks 2 and 3 (0.993 and 0.992, about 59 %) both cover the tile, mask 3 missing a thin sliver along the edge. Because det0 covered 72 % of the frame, below the 80 % needed for the frame-filling hypothesis, the separate-object rules applied and mask 2 became object 18_1. The complete call took 3.0 s on the laptop GPU.



(a) SAM mask 1 (b) SAM mask 2 (c) SAM mask 3 (d) RGBA cut-out 18_1

Figure 9. SAM 2.1 candidate masks and the saved cut-out.

(a–c) The three candidate masks for box det0; (d) the RGBA cut-out saved as object 18_1. Mask 2, with a predicted IoU of 0.993, was selected.

4.3.4 Object extraction and saving

The selected mask is upsampled to full resolution and the object is cut out along its bounding box, padded by 3 %. For the dataset, `segment_and_save()` in `runner.py` writes the binary mask, a transparent RGBA cut-out, a white-background copy, a preview and a JSON file with the boxes, scores, flags and decision log. Objects are named after the photograph, <photo id>_<n>; 18.jpg produced a single object, 18_1, saved as a 1,212 × 1,192 pixel cut-out (Figure 4d). These cut-outs are the input to every later stage: colour analysis, attribute prediction, the review application and the search index. For an uploaded photograph the same segmenter runs inside `mm/query.Segmenter`, which hands the RGBA crops straight to the application.

Two limitations remain. The low detection threshold produces many spurious boxes in busy photographs, so the result depends on hand-written selection rules. CLIP's notion of "ceramic" is also imperfect: a comment in the code records that white and cream glazes are sometimes scored as paper or cloth, which is why the primary object of a photograph is only flagged, never deleted, on the CLIP score alone. Errors that survive are handled by human review (Section 4.3.4).

4.3.5 Colour Extraction and Region Analysis

Colour is described twice in the implementation, by two different algorithms with different jobs. A ColorThief palette gives the three dominant colours of each object; it is stored with the dataset and shown on the review and result cards. Colour regions, built from SLIC superpixels and region-adjacency merging, describe where each colour sits on the object and how much it matters,

and these regions are what the colour similarity in the search compares. Both are converted to CIELAB and compared with the CIEDE2000 colour difference.

Region analysis with SLIC and RAG merging

Region analysis is implemented in `analyse()` in `mm/regions.py` with scikit-image [8]. Its input is the RGBA cut-out of one object, and it proceeds in five steps:

1. Pixels with alpha ≥ 125 form the object mask. The cut-out is resized so that its longer side is at most 320 pixels and converted to CIELAB.
2. SLIC (simple linear iterative clustering) [9] places about 220 seeds on a grid inside the mask and clusters pixels in a joint colour and position space with the distance of Equation (1). The result is a set of small superpixels that follow colour boundaries.
3. A region adjacency graph (RAG) [10] links neighbouring superpixels, and each edge carries the CIE76 distance between their mean Lab colours. `merge_hierarchical` repeatedly merges the closest pair while that distance is below 11.
4. Groups smaller than 1.2 % of the object are absorbed by a neighbour; a region counts as meaningful from 4 % of the area.
5. Each region receives its median Lab colour, a HEX code, the nearest reference shade name (by CIEDE2000, `glazeprep/appearance.py`), its area share, its centroid in 0–1 box coordinates with a position word, its distinctiveness (ΔE_{00} to the rest of the object), its contrast and an automatic importance (Equation (2)).

The SLIC distance and the importance are

$$D = \sqrt{d_c^2 + \left(\frac{d_s}{S}\right)^2 m^2} \tag{1}$$

$$I = 0.45\,min\left(1, \frac{\delta}{25}\right) + 0.35\sqrt{min\left(1, \frac{a}{0.25}\right)} + 0.20\,min\left(1, \frac{\kappa}{30}\right) \tag{2}$$

where d_c is the Lab colour distance and d_s the spatial distance between a pixel and a cluster centre, S the grid step, $m = 10$ the compactness, δ the distinctiveness, a the area share and κ the contrast of a region. The importance is multiplied by 0.3 for regions flagged as highlight or shadow. Small speckles are detected separately ($|L - median_{9\times9}(L)| > 14$) so that they do not fragment the regions. The output is the list of regions and a label map. Up to eight regions per object, each with

its Lab colour, centroid and weight, are stored in the search index and used by the colour similarity (Section 5.1.4).

On the 18_1 cut-out SLIC produced 210 superpixels, the RAG had 632 edges, merging left 63 groups and absorption left 14 regions (Figure 5). Table 2 lists the five most relevant. The three largest regions are warm browns and yellows that together cover 67 % of the tile, but the most important region is R4, the grey streak at the top left. It covers only 7.7 % of the object, yet it differs from everything else by $\Delta E_{00} = 22.8$, which gives it an importance of 0.76. Importance, not area, decides how much a region counts when two objects are compared. The tile also carries 104 small dark specks (#858A80, green-grey).

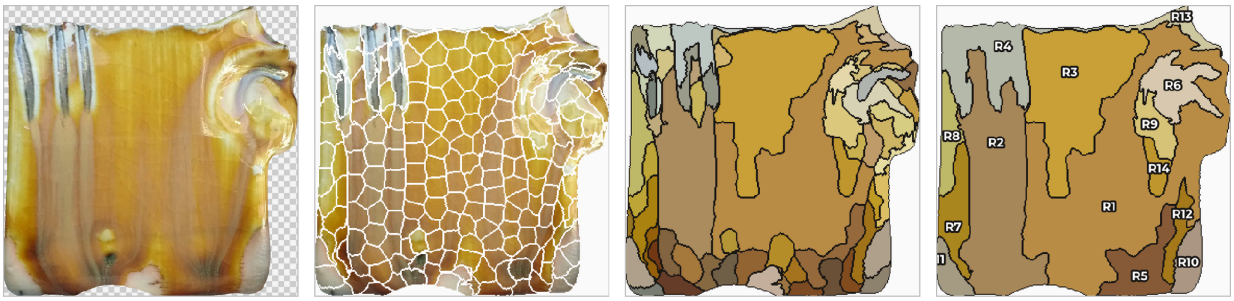


Figure 10. Colour regions of object **18_1.**

(a) Cut-out resized to 320 pixels; (b) 210 SLIC superpixels; (c) 63 groups after RAG merging below CIE76 = 11; (d) the 14 final regions after small groups are absorbed. Output of `regions.analyse()`.

Table 2. The five most relevant colour regions of 18_1.

There are 14 regions in total. R1–R3 are the largest; R4 is the most important.

Region		HEX	Name	Area	Position	Distinct. ΔE_{00}	Importance
R1		#BA8F4E	tan brown	33.7 %	center	6.3	0.55
R2		#A8885D	tan brown	17.6 %	left	7.7	0.51
R3		#C8A338	mustard yellow	15.8 %	top	10.2	0.55
R4		#B4BCB3	ash gray	7.7 %	top-left	22.8	**0.76**
R5		#885E35	earth brown	4.4 %	bottom-right	21.5	0.69

ColorThief dominant colours

The palette is produced by `scripts/17_colorthief_colors.py` with the ColorThief package [11]. *Input.* The transparent cut-out. *Processing.* ColorThief 0.2.1 implements modified median-cut

quantisation (MMCQ), a variant of Heckbert's median cut [12]: the colour box of the pixels is split repeatedly along its longest axis until six boxes remain (colour count 6, quality 5, image at most 600 pixels). Transparent pixels (alpha < 125) and near-white pixels are ignored. The first three colours are kept, each with the share of object pixels closest to it, and named after the nearest reference shade by CIEDE2000. *Output.* A three-colour palette with HEX code, name and share, written to the object's tag file and to `full_auto_tags.csv`. *Next stage.* The palette is shown on review and result cards, summarises the photograph in the recipe search mode, and feeds an automatic colour-relevance rule that is kept for the later evaluation. ColorThief keeps its own order, so the first colour is not necessarily the largest.

For 18_1 the stored palette is ochre yellow #B59047 (61 %), ash gray #CFD2C4 (8 %) and sand #B6AB8A (14 %), as shown on the review card in Figure 6. When the test image was uploaded to Glaze Search (Section 5.1.2), `image_feats.palette_lab()`, which calls the same function, returned #B59047 (61 %), #D0D3C4 (7 %) and #B6AB8B (14 %). The dataset and the application therefore agree.

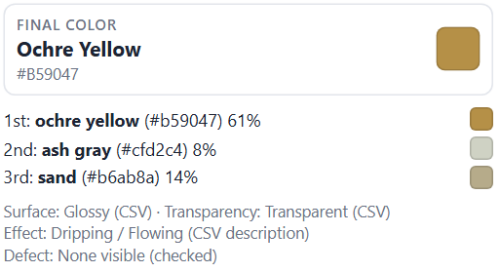


Figure 11. Stored ColorThief palette of **18_1.**

Review card with the final colour and the first three palette colours and their shares.

4.4 Training Process

Once the final training dataset was created after merging the extracted color name and hex code (training/training_data.csv), we trained our oxide and text projection heads, shared latent space, and performed clustering. Because the size of our dataset is relatively small and only text is used at this stage, each round of training only took a few minutes.

4.4.1 Shared Latent Space

The shared latent space enables cross-modal retrieval between ceramic glaze oxide compositions and descriptive text attributes. Since users typically search using properties such as color, surface finish, and firing cone rather than oxide compositions, the system learns a common representation that allows text-based queries to be compared directly against glaze chemistry.

The system consists of two parallel encoding streams:

1. **Oxide stream:** Each glaze is represented by 26 oxide composition features. These are normalized into relative proportions, square-root transformed, and reduced to 16 principal components using Principal Component Analysis (PCA). The explained variance from the 16 components was 93.5%. The resulting features are passed through a trainable linear projection head to produce a 256-dimensional embedding.
2. **Text stream:** Color name, surface, firing cone, transparency, and firing atmosphere, are combined into a single text string.

   ```
   f"color: {get('color_name')}, "
            f"Cone {get('Cone')}, {get('Surface')}, "
            f"{get('Transparency')}, {get('Atmospheres')}")
   ```

   A pretrained MiniLM Transformer converts this description into a 384-dimensional embedding. A separate trainable linear projection head maps this embedding into the same 256-dimensional space.

Both projection heads apply L2 normalization, allowing the similarity between text and chemistry embeddings to be measured using cosine similarity.

The two projection heads are trained using a symmetric InfoNCE (Information Noise Contrastive Estimation) loss. The objective encourages embeddings from matching text-chemistry pairs to have higher cosine similarity than non-matching pairs within a training batch.

The loss is calculated in both directions: text-to-chemistry and chemistry-to-text. This encourages bidirectional alignment between the two modalities.

During training, the pretrained MiniLM encoder remains frozen, while the oxide and text projection heads are optimized. Once training is complete, the shared projection head is saved to shared_space_heads.pt and the embedding vectors are saved to glaze_retrieval_vectors.npz. These files are used in retrieval later.

4.4.2 k-means Clustering

k-means clustering is incorporated as an unsupervised learning technique to identify groups of ceramic glaze recipes with similar chemical compositions. Its primary purpose is to organize glaze recipes into chemical families and support diversification of retrieval results.

k-means operates exclusively on the oxides of each recipe, independently of its color, surface finish, firing cone, and text descriptions.

The number of clusters was selected by comparing silhouette scores across candidate values of k. In the experiment, values of k = 5, 8, 10, 12, 15, and 20 were evaluated. k = 12 achieved the highest sampled silhouette score of approximately 0.144 and was selected for the clustering model. The clustering results are added to the original input dataset in csv format.

However, k-means assumes that clusters are relatively compact in the chosen feature space. The relatively low silhouette score suggests that the chemical groups overlap and are not strongly separated. Due to the dataset size, spotty availability of tagging information from crowdsourced recipes, and non-expert chemistry knowledge, this was a blocker we encountered during this project.

After k-means clustering is performed, a full catalog of recipes with cluster labels and other metadata is saved to glaze_retrieval_catalog.csv. This file is used in retrieval later.

4.4.3 Experimentation

Prior to implementing clustering with k-means, we experimented with using DBSCAN. However, the clustering results were not ideal for several reasons:

1. Highly imbalanced clusters: Approximately 73% of recipes were assigned to one large cluster, making it difficult to distinguish different chemical families.

2. High noise percentage: Around 24% of recipes were classified as noise (-1), meaning they were not assigned to any cluster.
3. Limited usefulness for diversification: Because most recipes belonged to one cluster or were classified as noise, DBSCAN provided limited information for selecting chemically diverse recommendations.

Ultimately, k-means produces a more balanced cluster distribution and assigns every recipe to a cluster. However, oxide similarity does not guarantee similarity in fired glaze appearance. Therefore, k-means is used as an optional diversification component rather than the primary retrieval mechanism.

4.4.4 K-means Result Diversification

K-Means clustering was used to group glaze recipes based on similarities in their oxide compositions. By default, these clusters do not affect the retrieval ranking, which prioritizes recipes that best match the user's query.

However, users can enable **Diversify mode** to explore alternative chemical formulations. Instead of returning only the highest-scoring recipes, the system initially selects a maximum of two recipes from each cluster. This allows users to discover glazes with different chemical compositions that may still achieve similar appearances. Although some diversified results may have lower similarity scores, they offer a wider variety of potential glaze formulations to explore.

4.5 Recipe Retrieval

In the user facing app/Python notebook, at inference time, a user’s photo provides the color and hex name to generate the user's text query. This query is encoded using MiniLM and the trained text projection head, then compared against the stored recipe embeddings using cosine similarity. This enables users to retrieve glaze recipes without providing any oxide information.

The surface and cone are **not hard filters**. They are used in the retrieval to find the most similar recipes, however surface and cone might not be an exact match in the results.

The retrieval results are subsequently reranked using additional descriptive attributes, including CIEDE2000 perceptual color similarity, surface finish, and firing cone compatibility.

$$S_{final} = 0.40S_{latent} + 0.40S_{color} + 0.10S_{surface} + 0.10S_{cone}$$

The final retrieval score is calculated using a weighted combination of four components: 40% from cosine similarity in the shared latent space, which captures the learned relationship between text descriptions and oxide compositions; 40% from CIEDE2000 perceptual color similarity, which measures how closely the recipe's color matches the user's requested color; and 10% each from surface finish and firing cone compatibility.

CIEDE2000 is a perceptual color difference metric that measures how visually different two colors appear to the human eye using the CIELAB color space. In our glaze retrieval system, it compares the user's requested hex color against recipe colors, where a lower CIEDE2000 (ΔE_{00}) value indicates greater color similarity.

Furthermore, chemical similarity does not guarantee similarity in fired glaze appearance. Therefore, k-means is used as an optional diversification component rather than the primary retrieval mechanism.

4.5.1 Retrieval Weight Experimentation and Results

Throughout the development of the ceramic glaze retrieval system, several retrieval strategies and weighting configurations were tested to improve the relevance of recommended glaze recipes.

Evolution of retrieval strategies

Experiment	Latent similarity	Hex color similarity	Surface	Cone
Initial InfoNCE retrieval	100%	—	—	—
Metadata-enhanced retrieval	70%	10%	10%	10%
Final color-weighted retrieval	40%	40%	10%	10%

Input image for all experiments

Figure 12. Sample glaze colour queries used for retrieval weight experiments: (a) midnight blue (#0931AA), glossy, Cone 6; (b) dusty pink (#C68896), glossy, Cone 6.

Experiment 1: InfoNCE-only retrieval

Initially, recipes were ranked solely using cosine similarity between the user's text embedding and the stored chemistry embeddings in the shared latent space.

Although retrieval performed better than random selection, the model struggled to consistently return visually relevant glazes. For example, a query requesting a midnight blue glaze frequently returned a variety of blue and non-blue colors.

This demonstrated that latent similarity alone was insufficient to capture the user's desired appearance accurately.

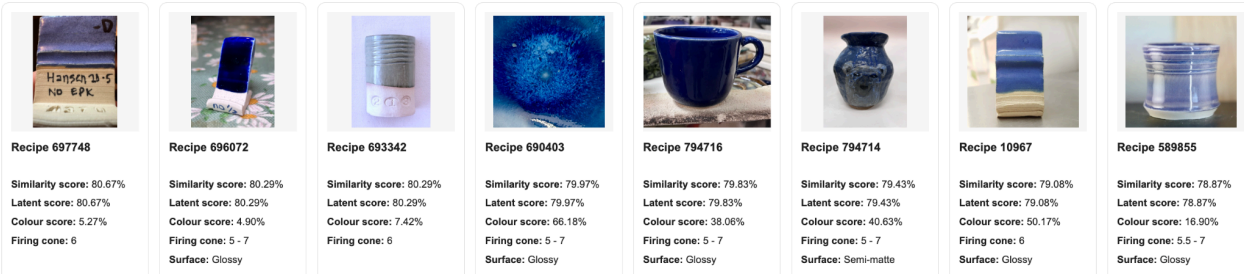


Figure 13. Results when only the latent space cosine similarity is used in retrieval for a midnight blue glossy cone 6.

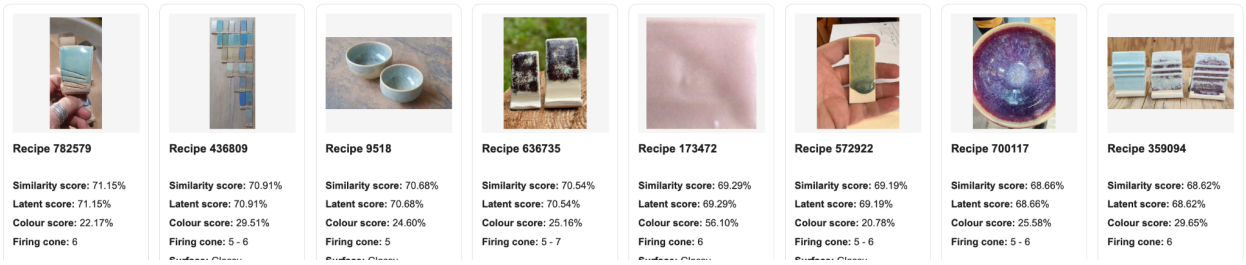


Figure 14. Results when only the latent space cosine similarity is used in retrieval for a dusty pink glossy cone 6.

Experiment 2: Metadata-enhanced retrieval (70/10/10/10)

To improve retrieval relevance, we introduced a weighted ranking function combining latent similarity (70%), CIEDE2000 perceptual color similarity (10%), surface finish (10%) and firing cone compatibility (10%).

For blue, the results looked decent. However for colors like pink, the exact shades of pink were still visibly different from the requested input. Additionally, the system initially retrieved only the top 200 candidates based on latent similarity before applying metadata reranking. This meant visually appropriate recipes could be excluded before their color similarity was evaluated.

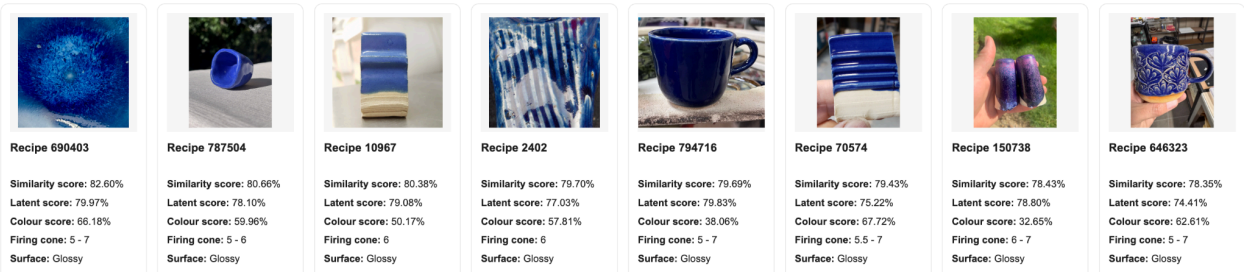


Figure 15. Results when the retrieval weights for midnight blue glossy cone 6 is 70/10/10/10.

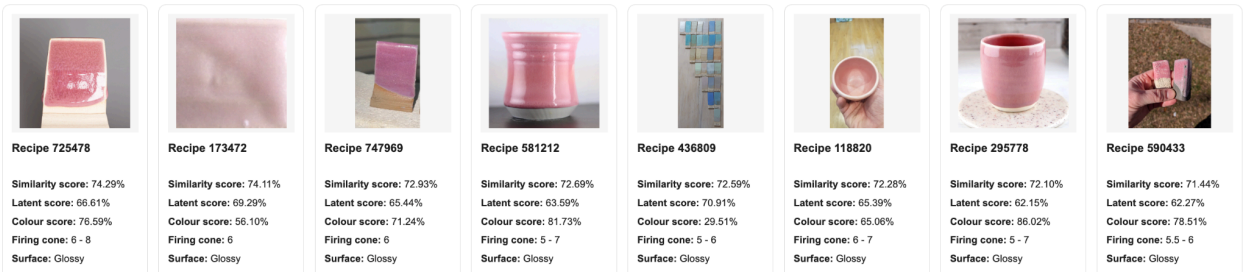


Figure 16. Results when only the retrieval weights for dusty pink glossy cone 6 is 70/10/10/10.

Experiment 3: Color-weighted retrieval (40/40/10/10)

To address these limitations, we modified the ranking weights to give equal importance to latent similarity and perceptual color similarity.

The updated scoring function is:

$$S_{final} = 0.40S_{latent} + 0.40S_{color} + 0.10S_{surface} + 0.10S_{cone}$$

Given the relatively small dataset size, we removed the top-200 candidate latent space restriction, allowing all eligible recipes to be evaluated using CIEDE2000. This change was intended to prioritize recipes that more closely matched the user's requested color while retaining the chemistry-text relationships learned through InfoNCE.

This weighting retrieved the most satisfactory results in terms of color matching and chemistry-text similarity.

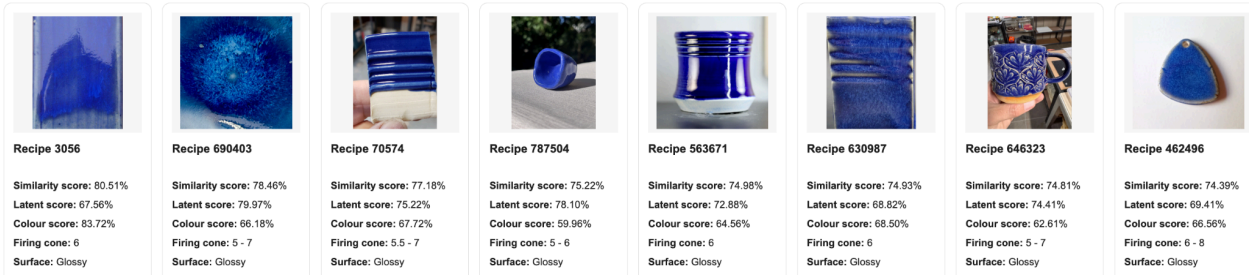


Figure 17. Results when the retrieval weights for midnight blue glossy cone 6 is 40/40/10/10.

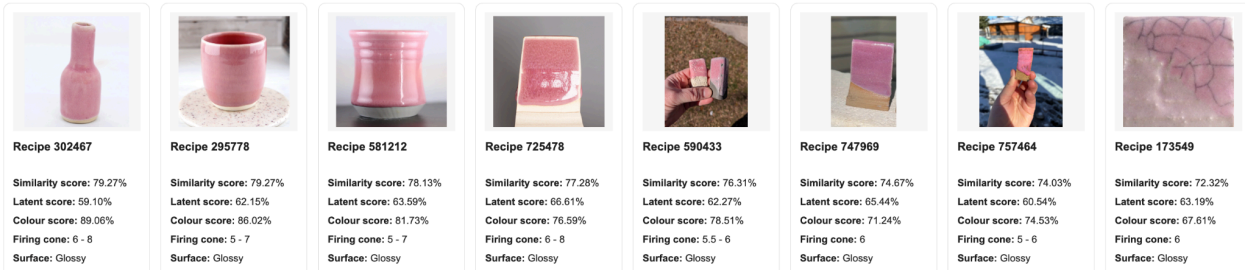


Figure 18. Results when the retrieval weights for pale pink glossy cone 6 is 40/40/10/10.

4.6 User facing app

There are two runnable prototypes for our project: a Streamlit app and a Python notebook. Both are run locally on the user's computer.

The code for the app is available in the repo in the **glaze_retrieval** folder**.**

The pre-trained models: Grounding DINO, SAM, MiniLM, and OpenCLIP are required to be downloaded locally. If the user does not have them pre-installed, they will be installed at the first run. The below screenshot shows a check in the Streamlit app for those models pre-installed:

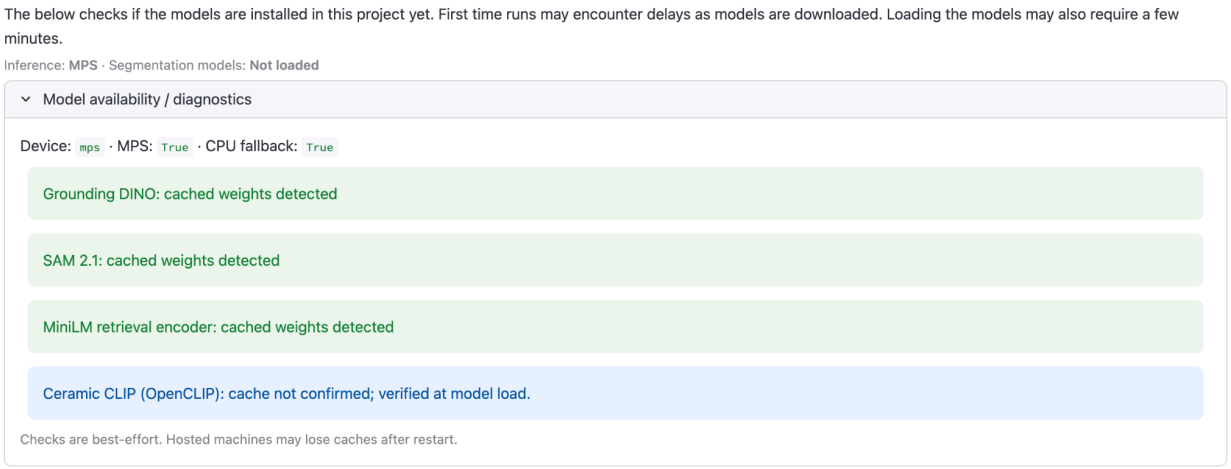


Figure 19. Streamlit application model availability diagnostics, showing the detection of cached pretrained model weights for Grounding DINO, SAM 2.1, and MiniLM, alongside OpenCLIP model verification.

The user is then required to upload an image of a pottery glaze they want similar recipes for.

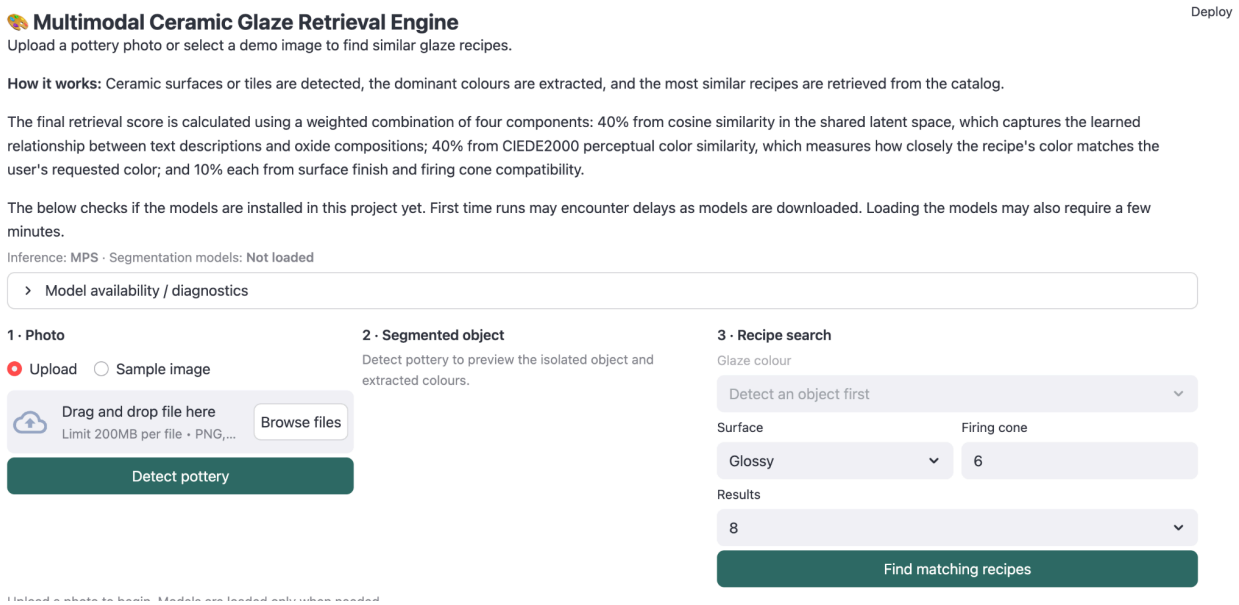


Figure 20. Streamlit user interface of the Multimodal Ceramic Glaze Retrieval Engine, showing the image upload, segmented object selection, and glaze recipe search components.

In the Python notebook, users should update the path to the image input.

2 · Choose an image

Set `IMAGE_PATH` to a local JPG, PNG or WebP file. Relative paths are resolved from the project folder. You can also use a file from `sample_images/`.

```
IMAGE_PATH = "sample_images/38571.jpg"  # <-- change this to your image path
path = Path(IMAGE_PATH).expanduser()
if not path.is_absolute():
    path = PROJECT_ROOT / path
if not path.is_file():
    raise FileNotFoundError(f"Image not found: {path}")
photo = ImageOps.exif_transpose(Image.open(path)).convert("RGB")
print("Image:", path.name, "·", photo.size)
display(photo.copy().resize((min(450, photo.width), round(photo.height * min(450, photo.width)/photo.width))))
```

✓ 0.0s

```
Image: 38571.jpg · (1600, 1200)
```

Figure 21. Jupyter Notebook implementation of the image input process, demonstrating image path configuration, file loading, and visualization of a sample ceramic glaze photograph.

4.6.1 Image upload and color extraction pipeline

When the user uploads an image, either on the Streamlit app UI, or by passing an image path in the Python notebook, the models are loaded and the undergoes the same Grounding DINO → OpenCLIP → SAM → ColorThief image processing and color extraction pipeline as we used for the training data.

The result of the image extraction and color extraction is shown to the user. In the situation where there is more than one pottery object detected, or the primary color is not actually the color of the glaze (it could be the unglazed base clay if the entire tile is not glazed), the user can select which color they want to use in the search.

The user can also select the desired surface, cone, and number of similar recipes to return.

In the Streamlit app:

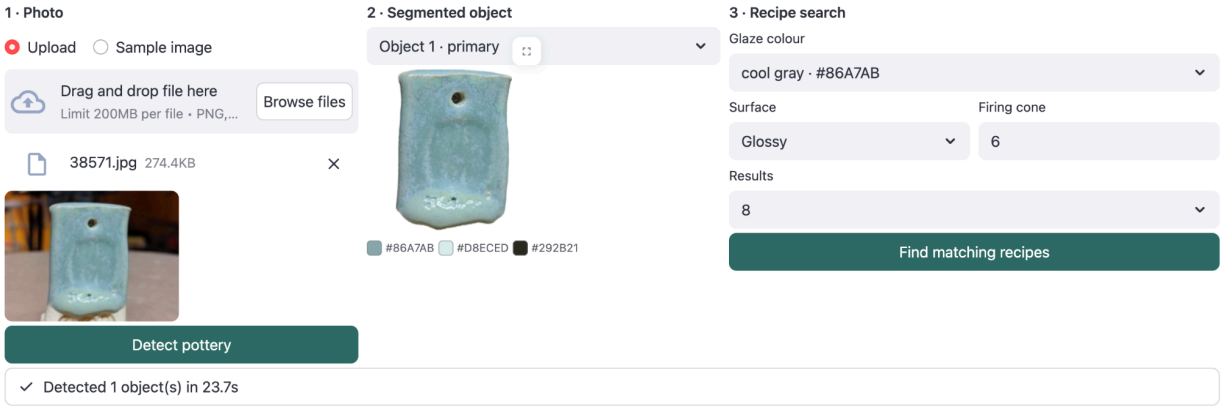


Figure 22. Streamlit app: Segmentation and color extraction results when only a single object is detected.

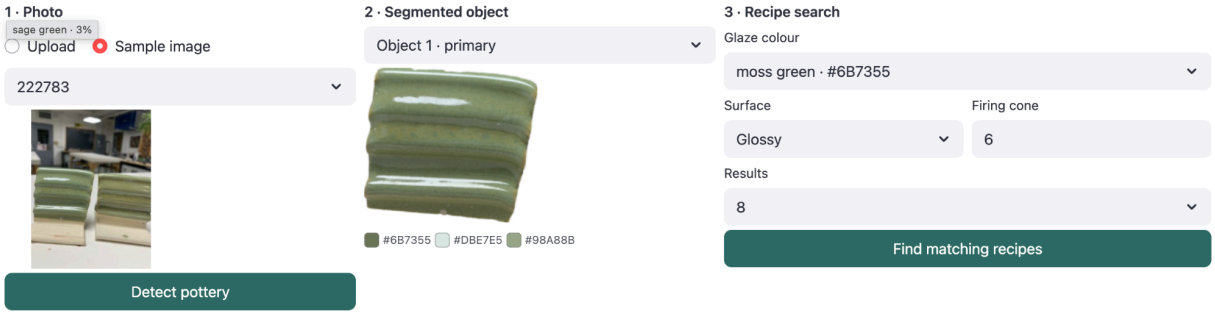


Figure 23. Streamlit app: Segmentation and color extraction results when more than 1 object is detected in the input photo. Object 1 is selected, and the color results from Object 1 are shown.

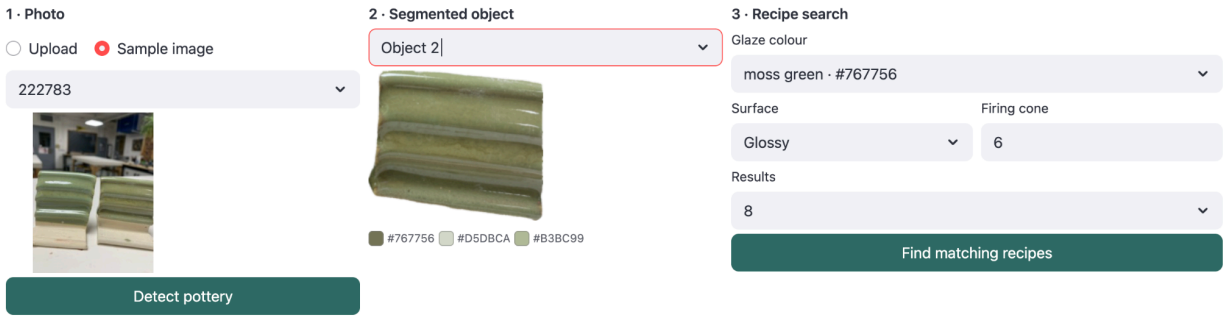


Figure 24. Streamlit app: Segmentation and color extraction results when more than 1 object is detected in the input photo. Object 2 is selected, and the color results from Object 2 are shown.

In the notebook version, the same object indexing and different color selection can be adjusted.

Set `OBJECT_INDEX` using the numbered detections above. `COLOR_INDEX` chooses which extracted colour is used for retrieval.

```
OBJECT_INDEX = 0  # <-- select detected object (0-based)
if not objects:
    raise RuntimeError("No objects detected; change IMAGE_PATH and rerun cells 2-3.")
if not 0 <= OBJECT_INDEX < len(objects):
    raise IndexError(f"OBJECT_INDEX must be between 0 and {len(objects)-1}")
selected_object = objects[OBJECT_INDEX]
palette_path = PROJECT_ROOT / "scripts" / "17_colorthief_colors.py"
spec = importlib.util.spec_from_file_location("gc2_palette", palette_path)
palette_module = importlib.util.module_from_spec(spec)
spec.loader.exec_module(palette_module)
palette, palette_note = palette_module.colorthief_palette(selected_object["image"])
print(palette_note or "Extracted colours:")
for i, item in enumerate(palette):
    display(HTML(f'<span style="display:inline-block;width:25px;height:20px;background:{html.escape(item["hex"])};border:1px solid
if not palette:
    raise RuntimeError("No colours were extracted from the selected object.")
COLOR_INDEX = 0  # <-- choose colour (0-based)
if not 0 <= COLOR_INDEX < len(palette):
    raise IndexError("COLOR_INDEX is outside the palette range")
selected_color = palette[COLOR_INDEX]
print("Selected:", selected_color)
```

✓ 0.3s

```
Extracted colours:
```

0: cool gray — #86A7AB

1: cool white — #D8ECED

2: soft black — #292B21

```
Selected: {'hex': '#86A7AB', 'name': 'cool gray', 'family': 'gray', 'fraction': 0.8477, 'delta_e00': 7.63, 'name_uncertain': False}
```

Figure 25. Notebook app: code snippet for color extraction.

5 · Retrieve regular and diversified results

Both result sets are computed **once**. Changing the display below does not rerun retrieval. The diversified list limits recipes per K-means cluster where possible.

```
SURFACE = "Glossy"    # e.g. Glossy, Matte, Satin
CONE = "6"           # firing cone
NUMBER_OF_RECIPES = 8  # 3 through 8
MAX_PER_CLUSTER = 2
if not 3 <= NUMBER_OF_RECIPES <= 8:
    raise ValueError("NUMBER_OF_RECIPES must be between 3 and 8")
import retrieval_service
params = dict(color=selected_color["name"], surface=SURFACE, cone=CONE,
              query_hex=selected_color["hex"], top_k=NUMBER_OF_RECIPES)
start = time.perf_counter()
regular_results = retrieval_service.retrieve_glazes(**params, diversify=False)
diversified_results = retrieval_service.retrieve_glazes(**params, diversify=True, max_per_cluster=MAX_PER_CLUSTER)
print(f"Retrieved both lists in {time.perf_counter()-start:.1f}s")
print("Regular IDs:", regular_results["recipe_id"].tolist())
print("Diversified IDs:", diversified_results["recipe_id"].tolist())
```

✓ 1.6s

```
Retrieved both lists in 0.3s
Regular IDs: [796559, 9792, 372428, 320383, 582781, 698589, 575426, 150981]
Diversified IDs: [796559, 9792, 372428, 320383, 582781, 86876, 73108, 414326]
```

Figure 26. Notebook app: code snippet for color retrieving

An input string consisting of color name, hex code, surface, and firing cone is created in the same format as our training Text Stream, and passed through the same MiniLM transformer → our

trained Text Projection Head → compared against the stored recipe embeddings using cosine similarity and the most similar 3-8 recipes are retrieved, as defined in the "Result Retrieval" section.

The original thumbnail image uploaded by the user for each recipe is displayed. These images are not stored in our system, and are fetched from the internet according to the image URL in the catalog.

The results are arranged based on decreasing similarity scores. There is also a link to the recipe on Glazy.

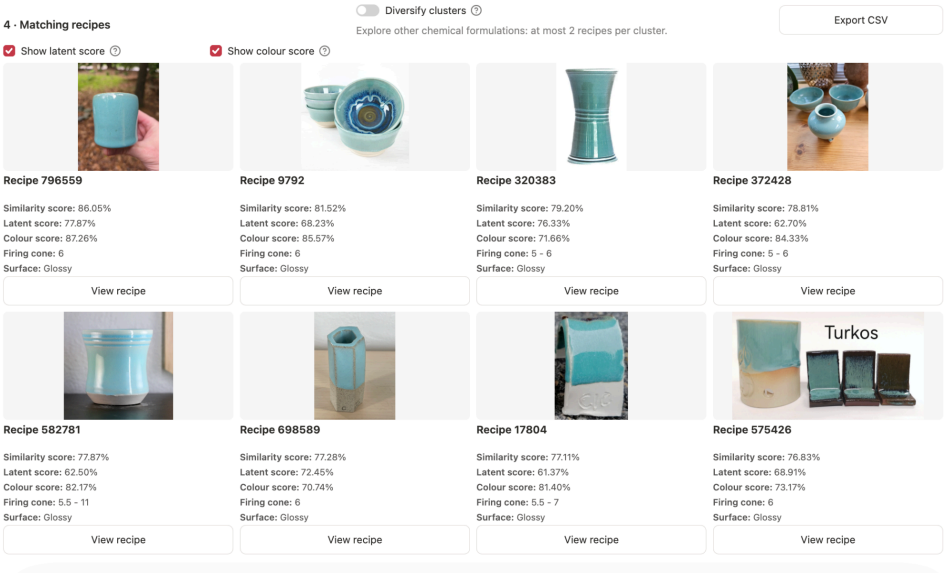


Figure 27. Streamlit app: Retrieved results for the sample input image.

The user can toggle between the regular and diversified clustering results.

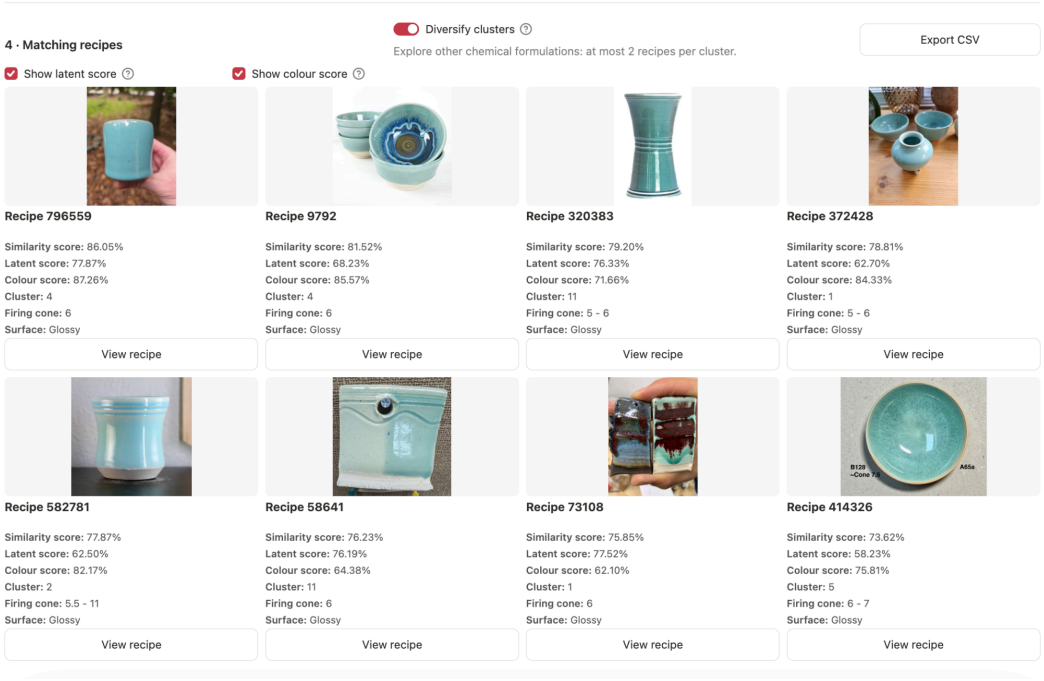


Figure 28. Streamlit app: Retrieved results with diversification for the sample input image. The highest ranking recipes with up to 2 results from each cluster are retrieved.

The same is available in the Python notebook version:

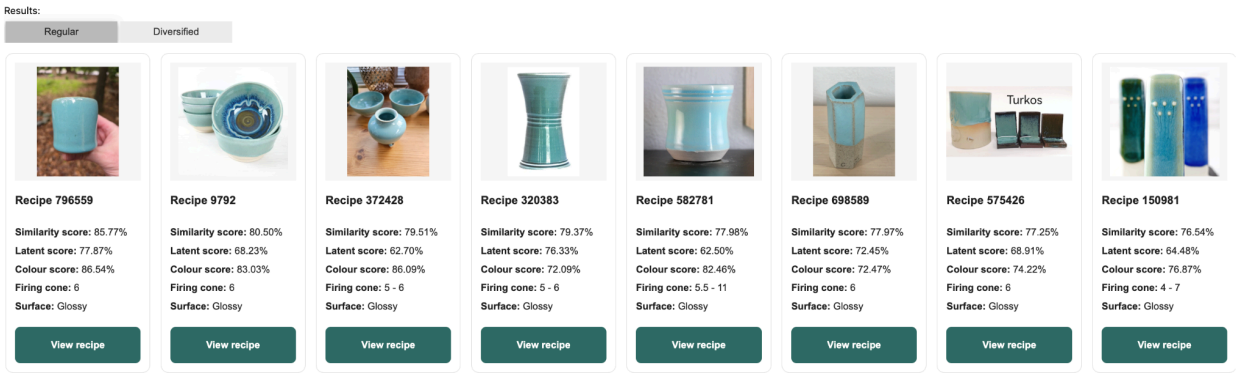


Figure 29. Notebook: Retrieved results for the sample input image (regular, non-diversified).

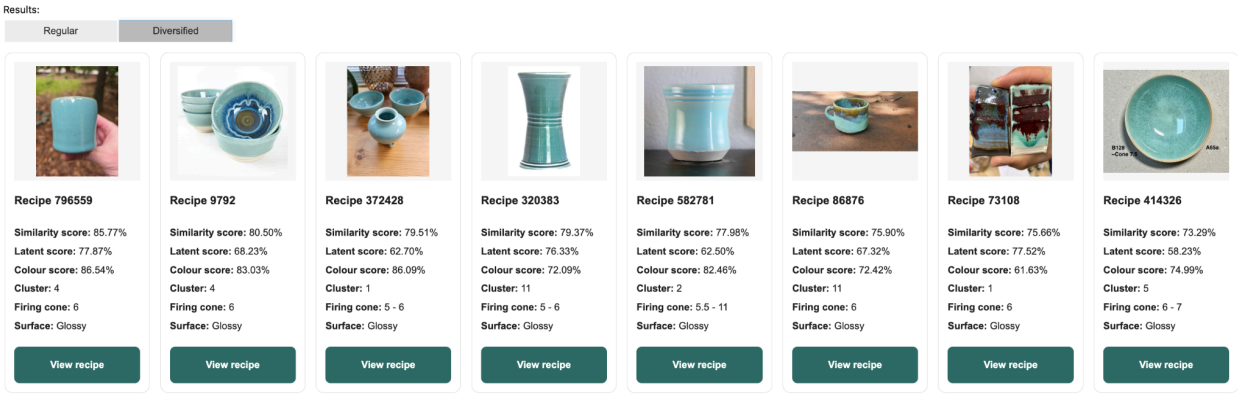


Figure 30. Notebook: Retrieved results for the sample input image (diversified using k-means cluster labels).

5. Evaluation Metrics

5.1 Real-Photo Retrieval Evaluation

The retrieval system was evaluated using 20 real ceramic photographs. Each photograph was searched using its extracted main colour, with the query tags set to “Glossy” and cone “6”. The top five recommendations were assessed for each photograph, producing 100 query–recipe pairs.

Four team members independently rated each recommendation on a five-point scale, resulting in 400 ratings. Assessment focused on the glaze’s colour, surface, and visual effects rather than the ceramic object’s shape or photographic background. A score of 1 indicated no match, 3 indicated a partial match, and 5 indicated an exact visual match under the evaluation rubric.

The four ratings were averaged for each recommendation. A recommendation was classified as relevant when its average score was at least 4. These judgements measure perceived visual suitability; they do not establish that a recipe will reproduce the photographed glaze after firing.

5.1.1 Hit@5

Hit@5 measures the proportion of queries with at least one relevant recommendation among the top five results.

Hit@5 = Number of queries with at least one relevant top-five result / Total number of queries

Of the 20 photographs, 19 had at least one relevant recommendation, giving a Hit@5 of **95%**. This indicates that the system usually provided at least one visually suitable candidate within the first five results.

5.1.2 Precision@5

Precision@5 measures the proportion of the top five recommendations judged relevant, averaged across queries.

Precision@5 = Average across queries of (Number of relevant top-five results / 5)

Across the 100 evaluated recommendations, 56 met the relevance threshold, giving a mean Precision@5 of **56%**. On average, each photograph received **2.8 relevant recommendations** among its top five results.

5.1.3 Recall@5

Recall@5 measures the proportion of all relevant recipes for a query that appear within the top five results.

Recall@5 = Number of relevant recipes retrieved in the top five / Total number of relevant recipes for the query

True Recall@5 could not be calculated because an independent reference set of relevant recipes was not available for these photographs. Rating only the retrieved results establishes their relevance but does not identify suitable recipes that the system failed to retrieve. Therefore, this evaluation reports Hit@5 and Precision@5 without presenting them as Recall@5.

5.1.4 Results and Discussion

Metric	Result
Test photographs	20
Recommendations evaluated	100
Raters	4
Individual ratings	400
Relevance threshold	Mean rating ≥ 4 out of 5
Hit@5	95%
Mean Precision@5	56%
Mean rating across all recommendations	3.89 / 5
Mean rating of the first-ranked recommendation	4.16 / 5

The high Hit@5 suggests that the system can help users identify promising recipes from real photographs. However, the lower Precision@5 shows that recommendation quality was not consistent across all five positions. Users would generally still need to compare the retrieved candidates before selecting recipes for further investigation.

The first-ranked recommendations received an average rating of 4.16, compared with 3.89 across all recommendations. This provides descriptive evidence that the top-ranked result was generally more suitable than the average retrieved result, although it does not establish that every result list was optimally ordered.

The evaluation also revealed limitations in image processing. For one teapot photograph, the segmenter selected the lid rather than the main body, causing the search colour to represent only part of the object. Such errors can influence retrieval before ranking takes place.

5.2 Colour-Matching Evaluation

A separate experiment assessed colour matching using 100 randomly generated, unique HEX colour queries with a fixed random seed of 42. Each query used “Glossy” and cone “6” as search settings. The first-ranked recipe was retrieved with diversification disabled, and its photograph’s primary object was used for colour measurement.

Colour difference was measured using CIEDE2000 (ΔE00), where lower values indicate a closer perceptual colour match. A photo colour difference of 5 or less was defined as a close match for this experiment.

Metric	Result
Random colour queries	100
Queries with usable measurements	100
Median photo colour difference (ΔE00)	9.41
Close matches (ΔE00 ≤ 5)	21 / 100 (21%)
Median percentage of object pixels within ΔE00 ≤ 10	24.9%
Distinct first-ranked recipes returned	61

All 100 queries produced usable measurements. However, only 21% met the defined close-match threshold, indicating that precise colour matching remains an area for improvement. The median colour difference of 9.41 also shows that successful retrieval does not necessarily imply a close colour match.

These results complement the real-photo evaluation but should be interpreted separately. The real-photo experiment assessed human judgements of overall glaze appearance across five recommendations, whereas this experiment measured colour differences for only the first-ranked result of randomly generated colour queries.

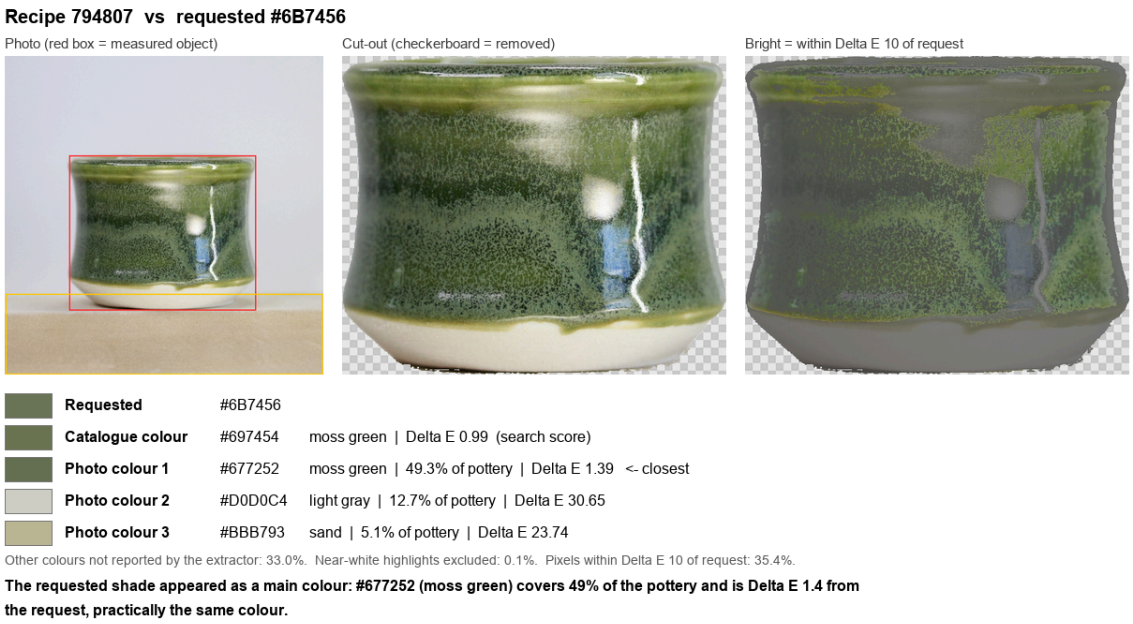


Figure 31. Colour-matching evaluation of ceramic glaze recipe 794807 against the requested colour (#6B7456), showing (a) the original photograph with the detected ceramic object, (b) the segmented glaze object, and (c) the pixel-level colour similarity visualization using the CIEDE2000 (ΔE00) metric.

5.3 Limitations and Future Evaluation

The real-photo evaluation used a small sample and ratings from the project team. Although averaging four ratings reduced dependence on a single evaluator, independent ceramic practitioners would provide stronger evidence of practical usefulness. Inter-rater agreement was not quantified in the reported results.

All real-photo queries used the same surface and firing-cone settings, limiting conclusions about performance under other conditions. Visual relevance was also assessed separately from firing compatibility; a visually similar recommendation should not be assumed to satisfy every technical requirement.

The random-colour experiment included colours that may not be represented closely in the recipe collection. Its results therefore reflect both retrieval performance and the colour coverage of the

dataset. Neither experiment validates the appearance of physically fired samples or comprehensively evaluates every combination of image, text, and chemical-formula inputs.

Future work should expand the test set, include independent raters, assess technical constraint satisfaction, and establish an independently labelled reference set for Recall@K. Comparisons against a simple colour-only baseline would also help determine the additional value of multimodal retrieval.

6. Mapping of System Functionalities against knowledge, techniques and skills of modular courses

Category	Implementation
Unsupervised learning	K-means clustering to group similar glaze recipes and diversify retrieval results
Machine learning	Train neural network projection heads using InfoNCE loss to align image and text embeddings in a shared latent space, enabling similarity-based retrieval
Deep learning	Grounding DINO for object detection, SAM 2.1 for segmentation, Ceramic CLIP for image classification, and MiniLM Transformer for text embeddings
Hybrid machine learning	Inputs from different modalities are used in separate models and the outputs from each are combined. Our retrieval algorithm combines weighted learned embedding similarity with CIEDE2000 colour similarity and rule-based matching of firing cone and surface finish to rank glaze recipes

Appendix A. Segmentation Steps and Thresholds

Table 3 lists every step of `CeramicSegmenterV3.__call__` with its settings, in the order in which the steps execute. Figure 2 in Section 4.3.1 shows the same steps applied to 18.jpg.

Table 3. Execution order and settings of `CeramicSegmenterV3`.

Steps of `__call__` in `ceramic_v3.py`; step 9 is `runner.segment_and_save()`.

Step	Model or rule	Input → output	Main settings	Status
1	Grounding DINO base	Photo + object prompt → boxes, scores, matched phrases	threshold 0.15; boxes < 0.15 % of the image dropped; keep ≥ 0.30 × best score; de-duplicate at box IoU ≥ 0.92; ≤ 40 boxes	Pretrained, frozen
2	OpenCLIP ViT-B/32, scene check	Whole photo → close-up / single / multi / non-ceramic probabilities	multi ≥ 0.50 switches to the multi-object regime	Pretrained, frozen, zero-shot
3	Seeding rule	Kept boxes → seed boxes	single scene: score ≥ 0.24 and ≥ 0.35 × best	Rule
4	SAM 2.1 Hiera-L	Photo (embedded once) + each kept box + one full-frame box → three masks per prompt	1,024-pixel encoder input; predicted IoU per mask	Pretrained, frozen
5	Mask selection	Candidate masks → object instances	A, one frame-filling object: box ≥ 80 %, mask ≥ 45 %, IoU ≥ 0.80. B, separate objects: IoU ≥ 0.55, ≥ 300 px, not background-like	Rule
6	Hand check: DINO → CLIP → SAM	"human hand. finger." boxes → p_{hand} per crop → hand mask subtracted	box ≥ 0.35; $p_{hand} \geq 0.50$; a hand mask with IoU ≥ 0.30 to an object is taken to be that object and ignored; otherwise at most 50 % of an object is removed	Pretrained, frozen
7	Clean-up	Instances → non-overlapping, split, de-specked instances	objects < 300 px dropped	Rule
8	OpenCLIP ceramic check	Masked crop of each object → $p_{ceramic}$	drop below 0.35 (single scene) or 0.20 (multi); flag below 0.35; the primary object is only flagged	Pretrained, frozen, zero-shot
9	Output	Full-resolution masks → mask, RGBA cut-out, white-background copy, preview, JSON metadata	bounding box padded by 3 %	`runner.segment_and_save()`

Appendix B. Model Versions and Processing Parameters

The values below were read from the source files named in the last column and from the metadata of the running application. They are collected here so that the main text can concentrate on what each step does.

Table 4. Model versions and processing parameters of the image pathway

Component	Version or setting	Source
Grounding DINO	IDEA-Research/grounding-dino-base, revision 12bdfa31; detection threshold 0.15	`scripts/setup_models.py`, `V3Params`
SAM 2.1	facebook/sam2.1-hiera-large, revision 665f8e2a; 1,024-pixel encoder input	`scripts/setup_models.py`, `V3Params`
OpenCLIP	laion/CLIP-ViT-B-32-laion2B-s34B-b79K, revision 1a25a446	`scripts/setup_models.py`
DINOv2	ViT-L/14 (`dinov2_vitl14_pretrain.pth`); masked mean of patch tokens → 256-number image head	`scripts/setup_models.py`, `mm/image_feats.py`
Colour regions	longest side 320 px; SLIC 220 segments, compactness 10; RAG merge below CIE76 = 11; regions < 1.2 % absorbed, meaningful from 4 %; specks $\lvert L - median(L)\rvert > 14$; up to 8 regions stored	`mm/regions.py`, `mm/balanced.py`
ColorThief	colorthief 0.2.1; colour count 6, quality 5, longest side 600 px; alpha ≥ 125, near-white skipped; first 3 colours kept	`scripts/17_colorthief_colors.py`
Colour matching	ΔE_{00} scale 40; position weight 0.5 (0 for "anywhere"); each direction capped at 2; importance high 1, normal 0.5, ignore 0	`mm/balanced.py`
Related colours	within $\Delta E_{00} = 28$ or the same family; common-name bonus 0.35; 18 returned	`mm/colorlab.py`
Duplicate check	64-bit dHash within 3 bits, then aspect ratio within 6 %, grey 32 × 32 correlation ≥ 0.95 and mean colour difference ≤ 14/255	`mm/data.py`, `mm/query.py`
Attribute layer v3	L2 logistic regression, $C \in \{0.01, 0.03, 0.1, 0.3, 1\}$; answered at 98 % validation precision; top 3 shown if $p \geq 0.05$ and ≥ 10 % of the best	`mm/attr_v3.py`, `mm/dino_calib.py`
Fusion	z-score per component over the searched gallery; missing similarity set to −1; photo 0.40, text 0.35, formula 0.25; ticked attributes take 40 % of the photo share	`mm/balanced.py`, `app/object_search.py`

Appendix C: User Installation Guide

This project runs locally on macOS or Windows.

- Streamlit (recommended): `glaze_retrieval_app.py` — upload an image or choose a sample, segment pottery, and search for recipes.
- Jupyter Notebook (alternative): `glaze_retrieval.ipynb` — specify an image file path and run the same segmentation/retrieval code in cells.

Both interfaces share `glazeprep/`, `segmentation_frontend.py`, `retrieval_service.py`, `retrieval_assets/` and `sample_images/`. Python 3.11 with a virtual environment is recommended. You only need one virtual environment for both interfaces.

macOS (Apple Silicon) — first-time installation

Creates the python virtual environment and installs requirements. In Terminal, `cd` to the extracted project folder, then:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip setuptools wheel
python -m pip install torch torchvision
python -m pip install -r requirements-macos.txt
python -m pip check
```

If using the Streamlit app, to start the app:

```
source .venv/bin/activate
export PYTORCH_ENABLE_MPS_FALLBACK=1
python -m streamlit run glaze_retrieval_app.py
```

If using the notebook:

```
source .venv/bin/activate
export PYTORCH_ENABLE_MPS_FALLBACK=1
python -m notebook glaze_retrieval.ipynb
```

Windows (PowerShell) — first-time installation

Creates the python virtual environment and installs requirements. Install Python 3.11, open PowerShell in the extracted project folder, then:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
```

If PowerShell blocks activation, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` and retry.

For NVIDIA CUDA, install the matching PyTorch wheels from https://pytorch.org/get-started/locally/ . Otherwise use:

```
python -m pip install torch torchvision
```

Then:

```
python -m pip install -r requirements-windows.txt
python -m pip check
python -m streamlit run glaze_retrieval_app.py
```

Or launch the notebook:

```
python -m notebook glaze_retrieval.ipynb
```

Notebook-only installation

If you only want the notebook, create and activate a Python 3.11 virtual environment as above, install PyTorch for your hardware, then:

```
python -m pip install -r requirements-notebook.txt
python -m notebook glaze_retrieval.ipynb
```

The notebook reads a local image path via `IMAGE_PATH`. Run cells in order and configure object, colour, surface, cone and result count. The regular and diversified recipe sets are both retrieved for comparison.

References

- Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms* (pp. 1027–1035). Society for Industrial and Applied Mathematics. https://theory.stanford.edu/~sergei/papers/kMeansPP-soda.pdf
- Au, D. (2026). *glazy-data: Public data from the Glazy database (export glazy_20260531)* [Data set]. GitHub. https://github.com/derekphilipau/glazy-data
- Digitalfire. (n.d.). *Digitalfire: Ceramic materials, chemistry, and glaze technology*. https://digitalfire.com
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining* (pp. 226–231). AAAI Press. https://www.dbs.ifi.lmu.de/Publikationen/Papers/KDD-96.final.frame.pdf
- Glazy. (n.d.). *Understanding glaze basics*. Glazy Help. https://help.glazy.org/concepts/glaze
- KyaniteLabs. (n.d.). *OpenGlaze* [Computer software]. GitHub. https://github.com/KyaniteLabs/openglaze
- Land, E. H., & McCann, J. J. (1971). Lightness and Retinex theory. *Journal of the Optical Society of America, 61*(1), 1–11. https://doi.org/10.1364/JOSA.61.000001
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory, 28*(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/
- Oord, A. van den, Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*. https://arxiv.org/abs/1807.03748
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems, 32*. https://arxiv.org/abs/1912.01703
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019* (pp. 3982–3992). Association for Computational Linguistics. https://aclanthology.org/D19-1410/
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7
- Sharma, G., Wu, W., & Dalal, E. N. (2005). The CIEDE2000 color-difference formula: Implementation notes, supplementary test data, and mathematical observations. *Color Research & Application, 30*(1), 21–30. https://doi.org/10.1002/col.20070
- Spherical Insights. (2026). *Top 15 companies in global glazes market (2026–2035): Spherical Insights analysis*. https://www.sphericalinsights.com/blogs/top-15-companies-in-global-glazes-market-2026-2035-spherical-insights-analysis
- Sentence Transformers. (n.d.). *all-MiniLM-L6-v2* [Pretrained language model]. Hugging Face. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
- Streamlit. (n.d.). *Streamlit documentation*. https://docs.streamlit.io/
- Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., & Zhou, M. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems, 33* (pp. 5776–5788). https://arxiv.org/abs/2002.10957
- Widjono, R. A., & Oktanuryansyah, M. N. (2023). User research of ceramics glaze database platform as instructional information: Study case Glazy.org. In *Proceedings of IMDES 2023* (pp. 345–359). Atlantis Press. https://doi.org/10.2991/978-2-38476-136-4_30
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP 2020: System Demonstrations* (pp. 38–45). Association for Computational Linguistics. https://aclanthology.org/2020.emnlp-demos.6/

AI Tool Declaration

We used ChatGPT, Claude and Gemini to generate ideas, draft and refine paragraphs, improve expression, check the report's claims against our code and results, and create charts and illustrations. We are responsible for the content and quality of the submitted work.